先进编译实验室
Advanced Compiling Laboratory
深度学习
编译器实战
--以Triton为例

编译实战培训班

讲师团介绍
先进编译实验室
ADVANCED COMPILING LABORATORY
李帅龙
• 先进编译实验室高性能计算部部长
• FlagGems&FlagTree开源项目贡献者
• 主研算子开发、性能优化
李嘉楠
• 先进编译实验室副主任
• 主研人工智能编译器、编译优化
• 多项开源AI项目负责人
• 2025智源大会Tech Tutorials技术分享者
王磊
• 先进编译实验室研究员
• B站《AI实操优化-Triton编译器》主讲人
• 主研深度学习编译优化
公众号：先进编译实验室

课程目录

- 课程环境配置&Triton与PyTorch
- Triton Kernel性能分析工具
- Triton语言特征详解及关键参数优化
- Triton编译器及Pass管理器
- Triton关键属性Layout介绍
- 编译优化实践(一)：代数变换pass添加
- 编译优化实践(二)：冗余去除pass添加

第一讲课程回顾

1、掌握环境配置使用方法

掌握远程服务器连接，以及conda虚拟环境配置方法

2、掌握torch.compile编译模式使用

了解torch.compile的使用方法，对简单的程序/函数进行编译执行

3、了解torch.compile编译的过程

通过查看简单程序/函数编译过程中的中间过程/表示，并进行对比查看，了解torch.compile编译的过程

4、掌握TorchBench模型基准测试框架的使用

安装Torchbench基准测试框架，使用Torchbench运行模型生成Triton Kernel

5、（拓展）了解整理一下目前哪些场景下使用了Triton算子

先进编译实验室 Advanced Compilation Laboratory

第二讲

Triton Kernel 性能分析工具

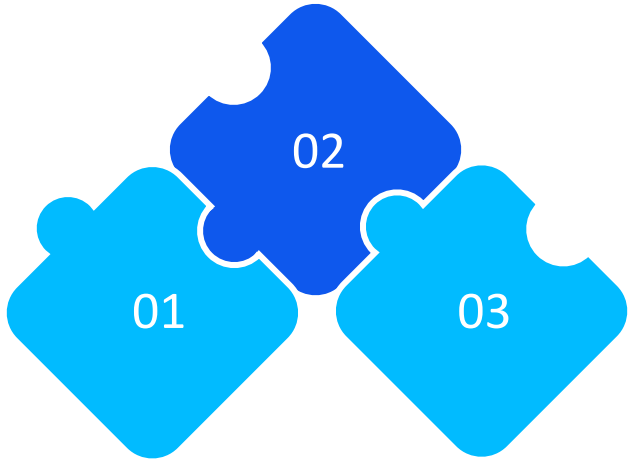
优化 Kernel 的常见步骤

检查GPU利用率等信息

例如检查Kernel是否使用了过多的寄存器，过高的寄存器使用会限制并行线程数，从而降低SM的利用率，细粒度定位每个Kernel的性能瓶颈

分析kernel的执行时间

统计并查看网络中各个Kernel的执行时间，找出执行较长的Kernel，粗粒度定位网络性能瓶颈。



优化Kernel的性能

根据性能瓶颈，通过调整线程执行结构、减少全局内存访问、提升内存访问模式效率等方法优化Kernel

常见的性能分析工具

PyTorch内置分析工具

torch.profiler 是 PyTorch 提供的性能分析工具，可以帮助开发者通过算子级别分析、内存分析、设备利用率分析、NCCL通信分析以及时间线可视化等方法，深入了解程序的执行情况，同时还可以通过**Holistic Trace Analysis（HTA）**工具将PyTorch Profiler收集的trace信息作为输入，进行一些可视化的分析工作

NVIDIA性能分析工具

NVIDIA CUDA生态下提供了多种性能分析工具，其中**Nsight Systems**是一款系统级的性能分析工具，用于分析应用程序的整体性能，可以帮助了解程序的执行流程、确定性能瓶颈以及发现并行化的机会；**Nsight Compute**是针对 CUDA 应用程序的内核级性能分析工具，专注于 GPU 上的计算性能，提供了详细的性能指标、诊断和指导，帮助开发者优化CUDA Kernel 的性能

Triton内置分析工具

Proton是为Triton设计的一款轻量级的性能分析器，能够提供了有关所调用GPU Kernel的程序上下文、元数据和硬件性能指标等信息

整网运行生成Kernel文件和profiler追踪文件

```
TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1 TORCHINDUCTOR_BENCHMARK_KERNEL=1 python -u
benchmarks/dynamo/torchbench.py --backend inductor --float16 --performance --only
BERT_pytorch --training --export-profiler-trace
```

- **生成整网Kernel文件**：通过 --training指定训练模式时，输出日志中包含两行编译模块存储路径，一行为前向图，一行为反向图；通过 --inference指定推理模式时，输出日志中包含一行编译模块存储路径
- **生成profiler追踪文件**：运行模型整网的基准测试时，可以通过添加—export-profiler-trace选项导出profiler的追踪文件
- **整网基准测试信息查看**：可以通过—output-directory="xxx"输出日志中的基准测试数据为csv文件，还可以通过tag控制输出内容

torch.profiler

单独研究每个编译模块的性能

```
python BERT_pytorch.py -p
```

- **时间及内存占用**
- **profiler追踪文件**
- **算子执行信息**：打印占用前十的算子，统计其CPU和GPU的时间及占用等信息
- **各类别Kernel信息**：包含pointwise、reduction、persistent_reduction以及cutlass/cudnn等各个类别算子在GPU上的执行时间、调用次数以及时间占比
- **GPU总繁忙时间占比**

查看追踪文件

- **CPU端事件**：前缀为triton_，该事件仅包含Kernel名称和启动，信息少于其他aten算子的Kernel启动，不包含输入shape、类型等信息
- **Kernel启动**：triton算子为cuLaunchKernel，区别与aten算子的cudaLaunchKernel
- **GPU端事件**：显示名称取决于unique_kernel_names的inductor配置
- **流事件**：GPU上每个kernel都是在CPU上运行的代码启动后发生的，可以查看GPU和CPU事件之间的连接
- **可视化地查看算子执行序列**
- **观察GPU利用情况**
- **查看算子并行、启动开销等情况**

torch.profiler

单独分析Kernel

```
torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
        torch.profiler.ProfilerActivity.XPU,],
    schedule=torch.profiler.schedule(
        wait=1,
        warmup=1,
        active=3,
        repeat=2,
        skip_first=0,
        skip_first_wait=0),
    on_trace_ready=trace_handler
    record_shapes=False,
    profile_memory=False,
    with_stack=False,
    with_flops=False,
    with_modules=False,
    experimental_config=None,
    execution_trace_observer=None,
    use_cuda=None,
    custom_trace_id_callback=None)
```

torch.profiler.profile：创建性能分析上下文，用来捕获CPU和GPU上的活动，包含以下参数选项

- activaties：用于指定想要捕获的活动
- schedule：用于设置采集的步骤
- on_trace_ready：用于指定性能追踪数据的保存处理方式
- record_shapes：记录输入张量的形状
- profile_memory：记录内存分配释放信息，用于分析内存泄漏或优化内存使用
- with_stack：记录算子的调用栈信息，用于帮助定位性能开销的代码片段
- with_flops：用于计算矩阵乘/2D卷积等特定算子的浮点运算次数FLOPs
- with_modules：记录与算子调用堆栈相对应的模块层次结构
- experimental_config：传入一些探索新特性的实验性配置选项
- execution_trace_observer：指定自定义的执行追踪观察器
- acc_event：是否保留和累积所有周期的事件数据，进行全局性能分析
- custom_trace_id_callback：设置一个在生成新跟踪 ID 时调用的回调

torch.profiler

单独分析Kernel

```
torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
        torch.profiler.ProfilerActivity.XPU,],
    schedule=torch.profiler.schedule(
        wait=1,
        warmup=1,
        active=3,
        repeat=2,
        skip_first=0,
        skip_first_wait=0),
    on_trace_ready=trace_handler
    record_shapes=False,
    profile_memory=False,
    with_stack=False,
    with_flops=False,
    with_modules=False,
    experimental_config=None,
    execution_trace_observer=None,
    use_cuda=None,
    custom_trace_id_callback=None)
```

torch.profiler.profile：创建性能分析上下文，用来捕获CPU和GPU上的活动，包含以下参数选项

- activaties：用于指定想要捕获的活动
- schedule：用于设置采集的步骤
- on_trace_ready：用于指定性能追踪数据的保存处理方式
- record_shapes：记录输入张量的形状
- profile_memory：记录内存分配释放信息，用于分析内存泄漏或优化内存使用
- with_stack：记录算子的调用栈信息，用于帮助定位性能开销的代码片段
- with_flops：用于计算矩阵乘/2D卷积等特定算子的浮点运算次数FLOPs
- with_modules：记录与算子调用堆栈相对应的模块层次结构
- experimental_config：传入一些探索新特性的实验性配置选项
- execution_trace_observer：指定自定义的执行追踪观察器
- acc_event：是否保留和累积所有周期的事件数据，进行全局性能分析
- custom_trace_id_callback：设置一个在生成新跟踪 ID 时调用的回调

torch.profiler

使用torch.profiler单独分析Kernel

```python
if __name__ == '__main__':
    from torch._inductor.runtime.benchmarking import benchmarker

    args = get_args()
    ms = benchmarker.benchmark_gpu(lambda: call(args), rep=40, fast_flush=True)
    num_gb = 0.201332736
    gb_per_s = num_gb / (ms / 1e3)
    print(f"{ms:.3f}ms    {num_gb:.3f}GB    {gb_per_s:.2f}GB/s")
```

```python
if __name__ == '__main__':
    args = get_args()
    torch.cuda.set_device(0)
    print("Warning up ...")
    for _ in range(10):
        call(args)
    torch.cuda.synchronize()

    with torch.profiler.profile(
        activities = [torch.profiler.ProfilerActivity.CPU,
                      torch.profiler.ProfilerActivity.CUDA],
        schedule = torch.profiler.schedule(
            wait=0,warmup=0, active=20, repeat=40
        ),
        record_shapes= True,
        profile_memory=True
    ) as p:
        for _ in range(20):
            call(args)
        torch.cuda.synchronize()
    print(p.key_averages(group_by_input_shape=True).table())
    p.export_chrome_trace("/root/autodl-tmp/BERT_pytorch/kernel_7.json")
```

HTA可视化工具

Holistic Trace Analysis (HTA)是一个面向PyTorch用户的开源性能分析和可视化Python库，HTA将PyTorch Profiler收集的trace信息作为输入，进行一些分析工作。

安装方式

```
pip install HolisticTraceAnalysis
```

使用方式

```
from hta.trace_analysis import TraceAnalysis
analyzer = TraceAnalysis(trace_dir = "/trace/folder/path")
```

https://github.com/facebookresearch/HolisticTraceAnalysis
https://pytorch.ac.cn/tutorials/beginner/hta_intro_tutorial.html

HTA可视化工具

使用方式

```
from hta.trace_analysis import TraceAnalysis
analyzer = TraceAnalysis(trace_dir = "/trace/folder/path")
```

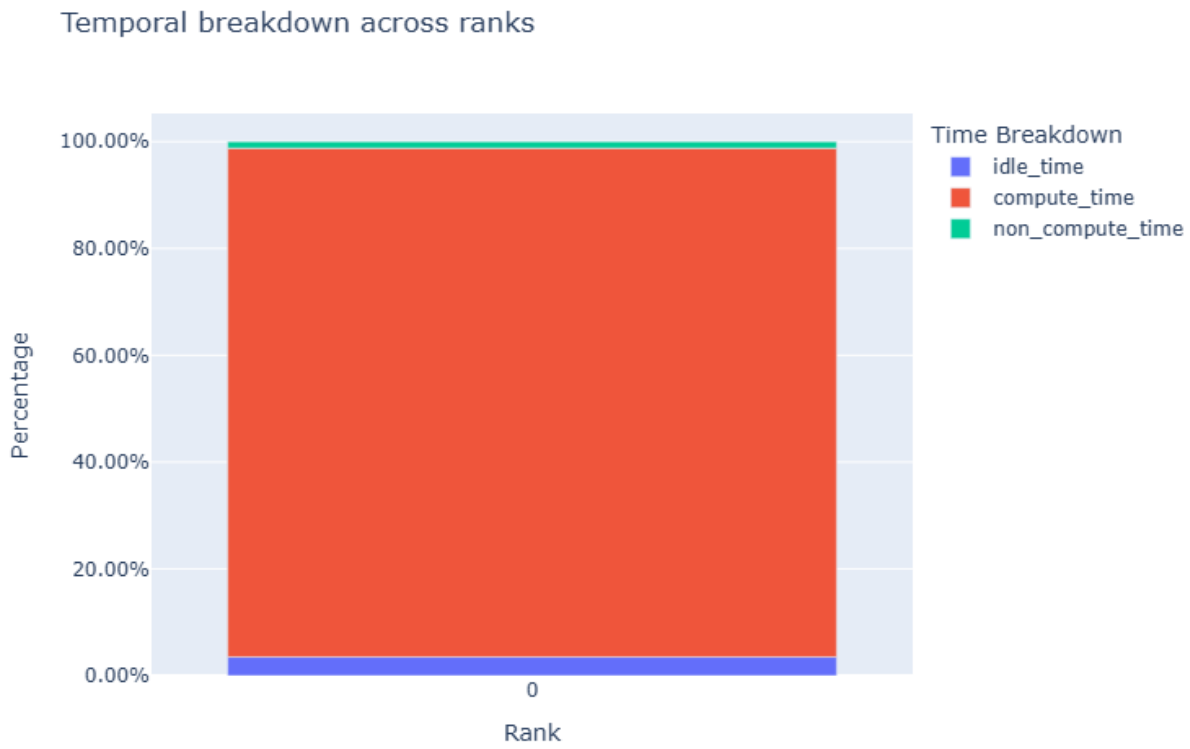
```
from hta.trace_analysis import TraceAnalysis
from glob import glob
import json

for trace in glob("/root/autodl-
tmp/BERT_pytorch/hta_trace/inductor_BERT_pytorch_rank_0.json"):
  with open(trace, 'r') as f:
    trace_obj = json.load(f)
    trace_obj['rank'] = 0

  with open(trace, 'w') as f:
    json.dump(trace_obj, f)

analyzer = TraceAnalysis(trace_dir="/root/autodl-
tmp/BERT_pytorch/hta_trace")
```

HTA可视化工具

时间分类

将时间划分为空闲时间、计算时间和非空闲时间，为实现GPU的高效利用，应最大限度地利用计算时间，最大限度地减少空闲时间和非计算时间

```
time_spent_df = analyzer.get_temporal_breakdown(visualize = True)
```



HTA可视化工具

空闲时间分解

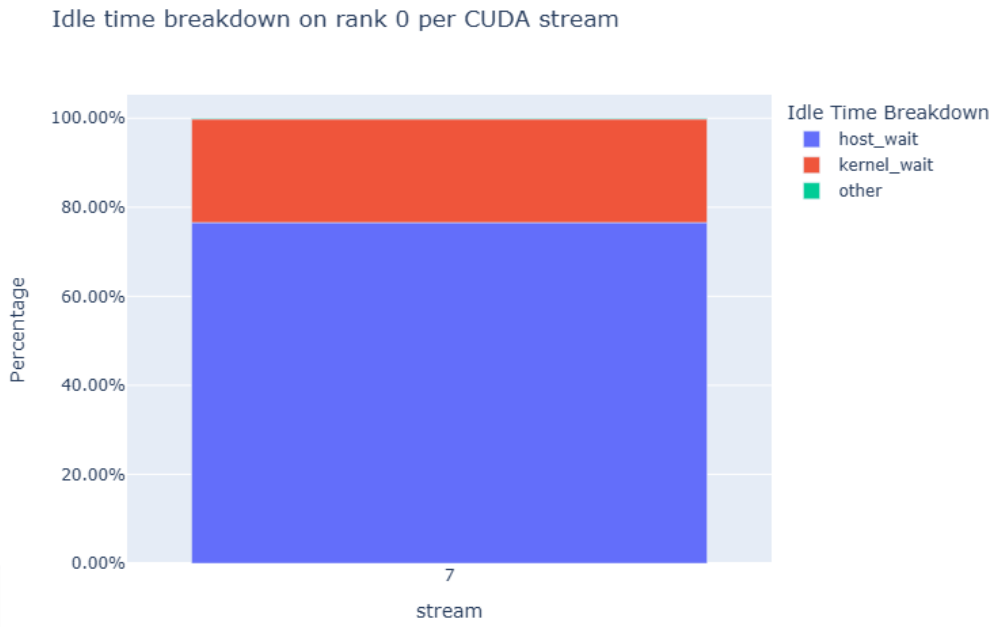
将空闲时间细分为主机等待、Kernel等待、其他等待

主机等待：指GPU空闲是由于CPU不够快的启动Kernel，可以去查看一下较慢的CPU算子、增大batchsize 或者进行算子融合，来减小此类问题

Kernel等待：指由于连续启动Kernel的短暂开销，可以通过启用CUDA Graph来减小此类问题

其他等待：由于信息不足而无法归类的等待原因，包括使用CUDA事件的CUDA流之间的同步、Kernel启动延迟等

```
idle_time_df = analyzer.get_idle_time_breakdown()
```

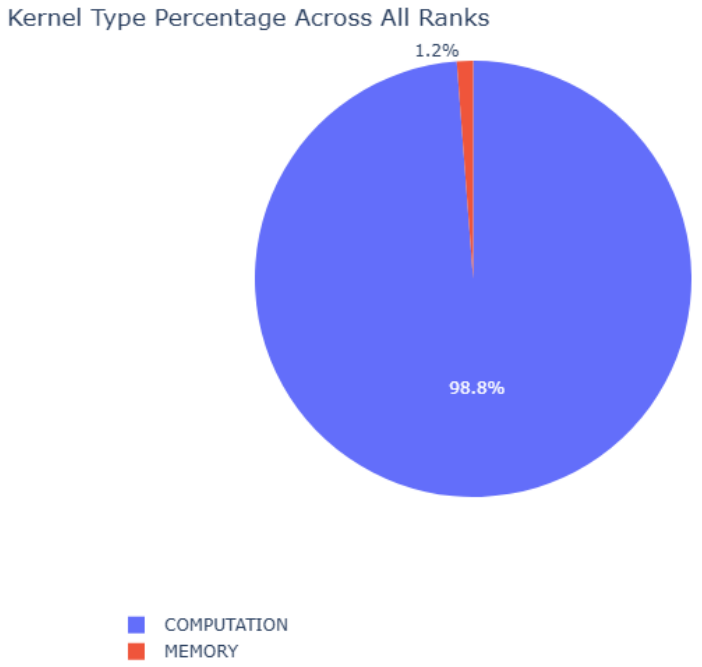


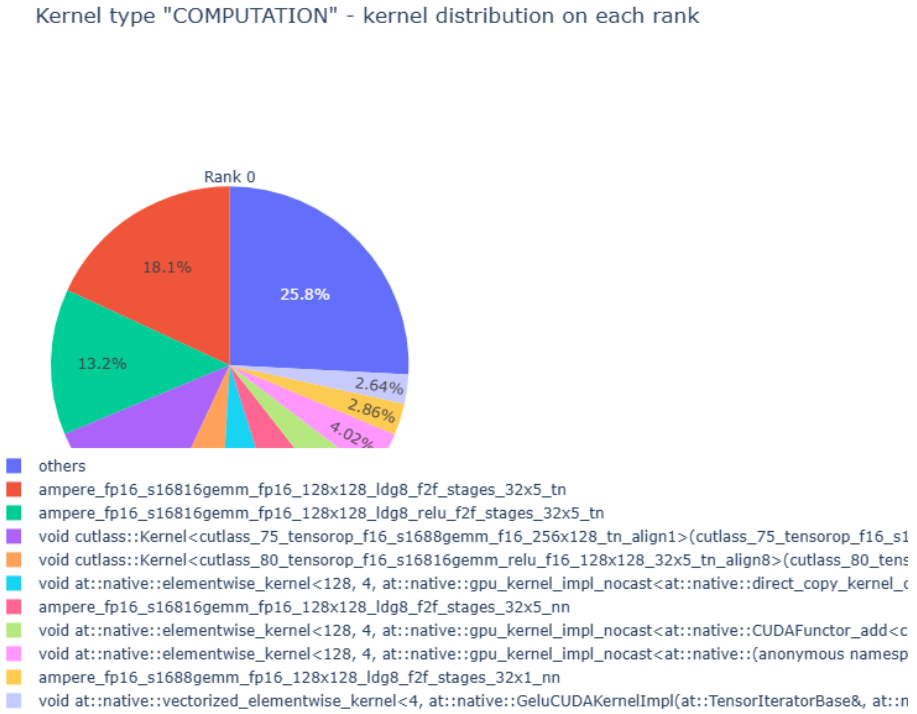
HTA可视化工具

kernel分类

将Kernel分为通信、计算、内存等类别，统计各个类别的时间

```
kernel_type_metrics_df, kernel_metrics_df = analyzer.get_gpu_kernel_breakdown()
```





HTA可视化工具

CUDA Kernel启动分析

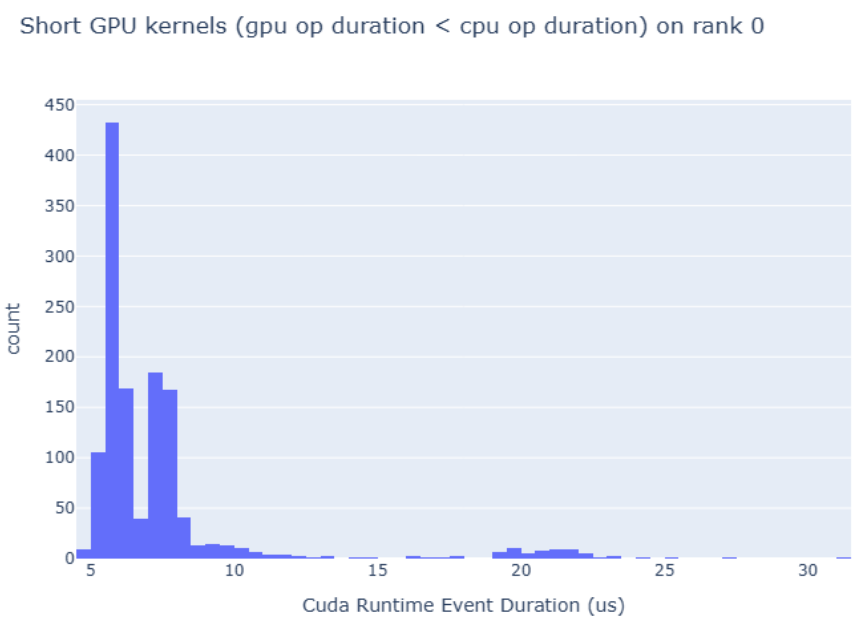
通过分析CPU算子、GPU Kernel以及启动延迟的时间可以得到以下信息

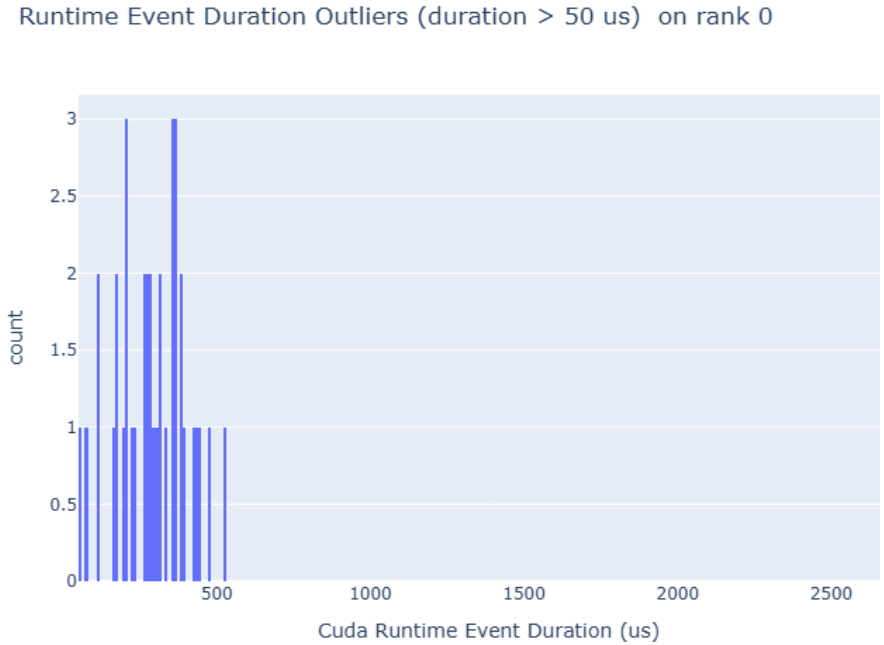
时间较短的GPU Kernel：指GPU Kernel执行时间小于CPU算子

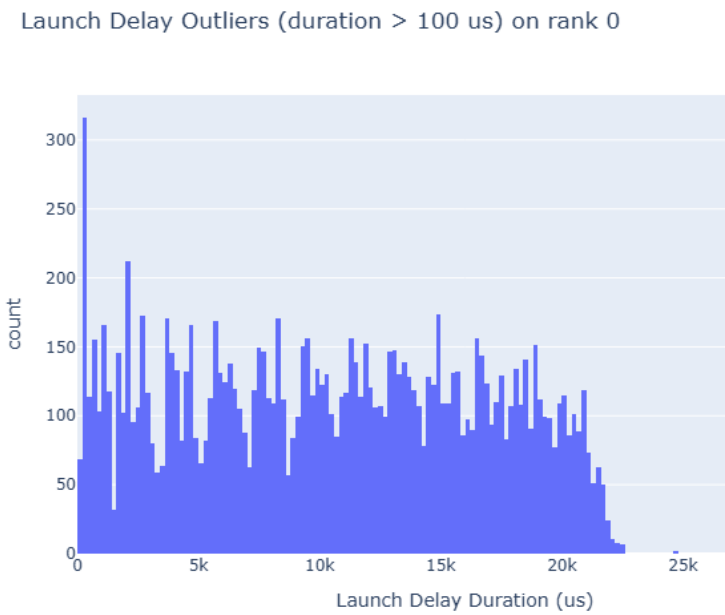
运行时事件异常：执行时间过长的CPU运行时事件

启动延迟异常：GPU Kernel调度需要较长时间

```
kernel_info_df = analyzer.get_cuda_kernel_launch_stats()
```







HTA可视化工具

频繁调用的CUDA Kernel序列

识别频繁调用的CUDA Kernel序列机器相应的CPU操作，可以考虑进行算子融合

```
cuda_sequences_df = analyzer.get_frequent_cuda_kernel_sequences(
    operator_name = "aten::linear",
    output_dir = "/tmp/"
)
```

内存带宽和队列长度计数器

内存带宽：通过内存复制 (memcpy) 和内存集 (memset) 事件来测量从 H2D、 D2H 和 D2D 复制数据时使用的内存复制带宽

队列长度：每个CUDA流上未完成的操作数

```
analyzer.generate_trace_with_counters()
```

此外还有其他的一些CUPTI计数器分析、关键路径分析、通信计算重叠分析等用于提供GPU Kernel加速、roofline分析以及其他优化的灵感，参见HTA项目主页https://github.com/facebookresearch/HolisticTraceAnalysis去进行查看

Nsight Systems VS Nsight Compute

Nsight Systems

Nsight Systems是一款系统级的性能分析工具，用于分析应用程序的整体性能，使用Nsight Systems可以了解应用程序的执行流程、确定性能瓶颈以及发现并行化的机会

主要偏向于对应用程序的整体流程进行分析

Nsight Compute

Nsight Compute是一款针对 CUDA 应用程序的内核（Kernel）级性能分析和调试工具，它专注于 GPU 上的计算性能，提供了详细的性能指标、诊断和指导，帮助开发者优化 CUDA Kernel 的性能。

主要偏向于对应用程序中Launch的所有Kernel进行详细的分析。

Nsight Systems & Nsight Compute

安装及使用

Nsight Systems和Nsight Compute都可以在服务器上通过命令行指令（CLI版本）进行profiling并生成相关的分析报告后，将分析报告下载到本地后使用对应的GUI版本进行分析报告内容的查看。

在官网上下载对应版本后，使用相关命令或者按照引导进行安装即可

DOWNLOADS
- Nsight Compute 2025.1.1 (Linux)
- Nsight Compute 2025.1.1 (Windows)
- Nsight Compute 2025.1.1 (MacOS-x64)
- Nsight Compute 2025.1.1 (MacOS-Arm64)
- Nsight Compute 2025.1.1 (ArmServer)

DOWNLOADS
- Nsight Systems 2025.1.1 (Linux Host .run Installer)
- Nsight Systems 2025.1.1 (Linux Host .deb Installer)
- Nsight Systems 2025.1.1 (Linux Host .rpm Installer)
- Nsight Systems 2025.1.1 (Linux CLI only .rpm Installer)
- Nsight Systems 2025.1.1 (Linux CLI only .deb Installer)
- Nsight Systems 2025.1.1 (Windows Host)
- Nsight Systems 2025.1.1 (macOS Host)
- Nsight Systems 2025.1.1 (Arm Server Full .deb Installer)
- Nsight Systems 2025.1.1 (Arm Server Full .rpm Installer)
- Nsight Systems 2025.1.1 (Arm Server Full .run Installer)
- Nsight Systems 2025.1.1 (Arm Server CLI only .deb Installer)
- Nsight Systems 2025.1.1 (Arm Server CLI only .rpm Installer)

https://developer.nvidia.com/tools-downloads

Nsight Systems

使用命令后会生成 .nsys-rep 文件

```
nsys profile --stats=true -o mat_test python matmul.py
```

使用 GUI 分析 .nsys-rep 文件

- **CUDA API**： Kernel启动和执行关系
- **GPU工作负载**
- **Libraries**：cuBLAS、cuDNN
- **OS 线程状态、CPU利用率、pthread、文件I/O**
- **NVTX**：通过在应用程序中添加标签来帮助理解timeline的执行过程

  python程序可以添加 with nvtx.range(<str>)

Nsight Compute

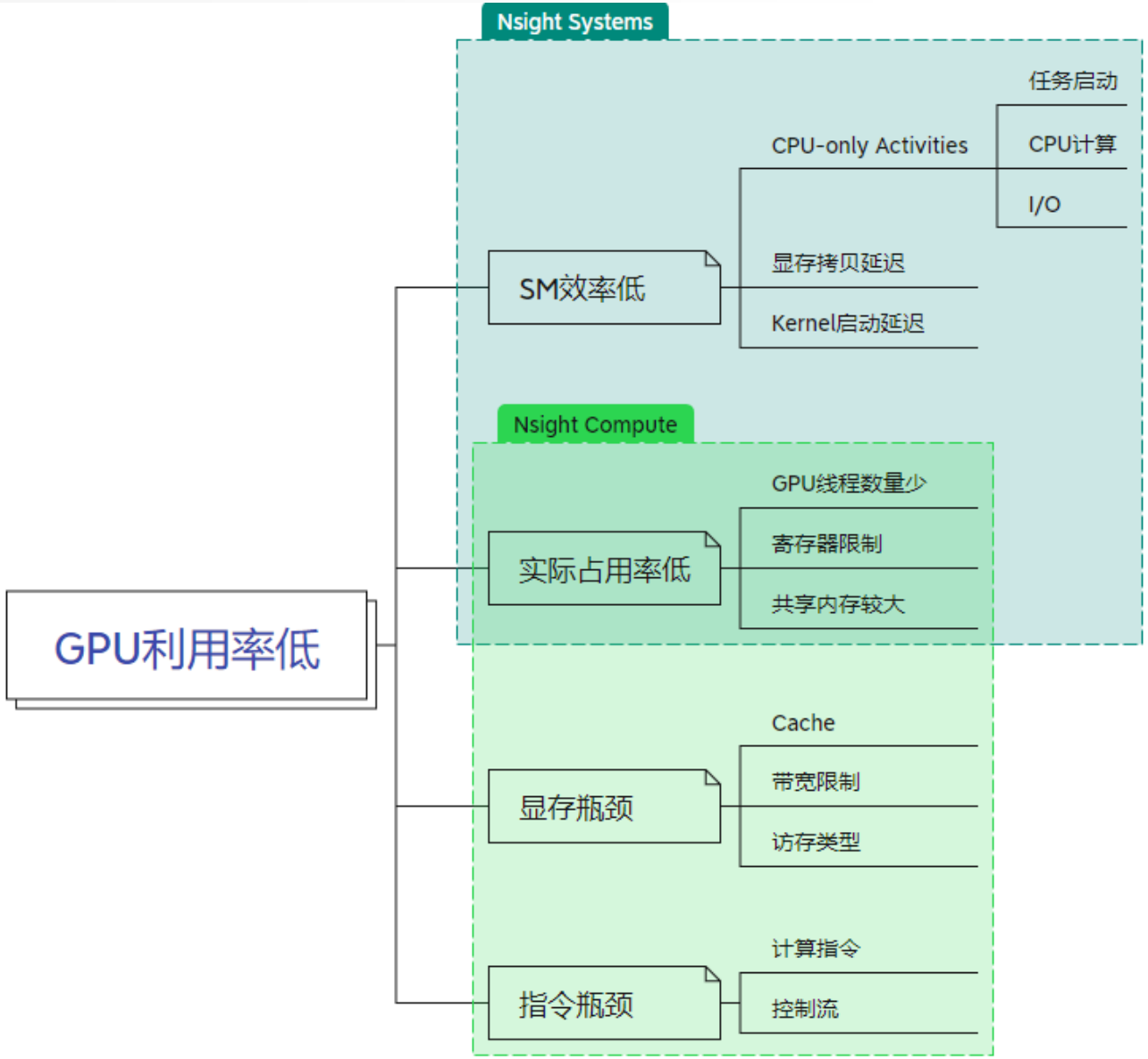
使用命令后会生成 .ncu-rep 文件

```
ncu --set full --target-processes all -o my_report ./your_program
```

使用 GUI 分析 .ncu-rep 文件

- **API Stream**： 进行交互式分析
- **GPU利用率**：SM 和 Memory 利用率
- **计算工作负载分析**：SM 计算资源的详细分析（每个时钟周期指令数以及pipeline利用率）
- **Warp调度器及Warp状态分析**
- **内存工作负载分析**
- **Kernel Launch 分析**
- **源码与指令映射关系分析**

Nsight Systems VS Nsight Compute



Proton

Proton是为 Triton 设计的一款轻量级的性能分析器，能够提供了有关所调用 GPU Kernel 的程序上下文、元数据和硬件性能指标等信息

- **对指定区域进行性能分析**

  proton.start(profile_name: str) -> session_id: int

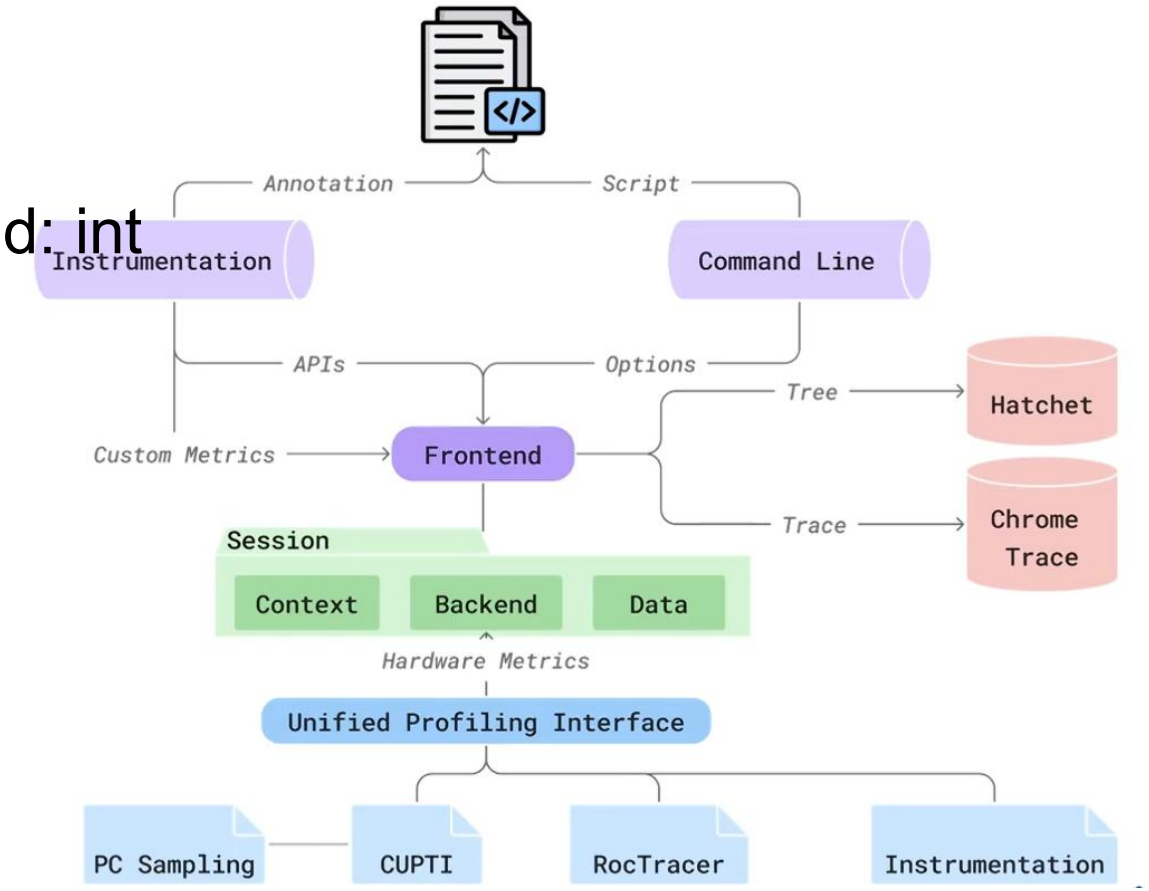
  proton.finalize()

- **用语义信息标注用户定义的区域**

  proton.scope(name: str, metrics: {})

- **自定义元数据**

  triton.jit(launch_metadata = metadata_fn)

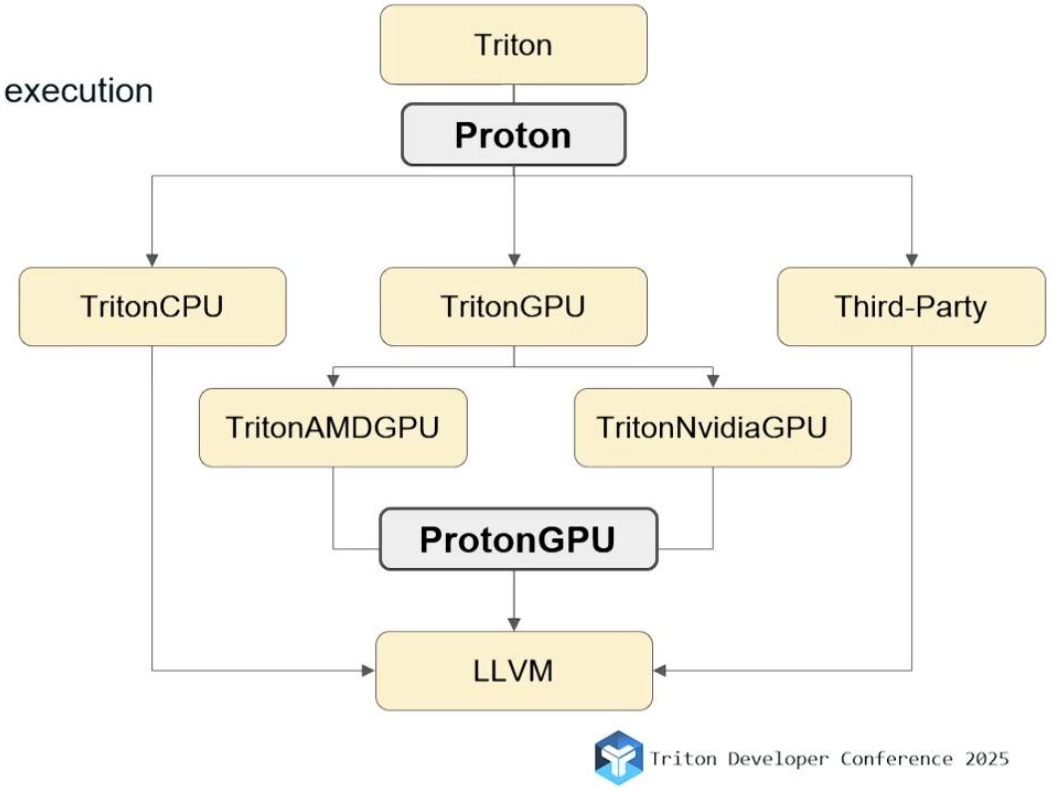


https://github.com/triton-lang/triton/tree/main/third_party/proton

Proton

IR Design and Triton Compiler Integration

- Requirements
  - Keep Proton IRs independent from Triton IRs
  - Compatible with Triton's existing IR model and execution model
- Proton IR
  - **Frontend annotation:** *proton.record*
- ProtonGPU IR
  - **Resource allocation:**
    - *proton_gpu.global_scratch_alloc*
    - *proton_gpu.segment_alloc*
  - **Runtime event recording:**
    - *proton_gpu.read_counter*
    - *proton_gpu.circular_store*
  - **Finalization:** *proton_gpu.finalize*



https://www.youtube.com/watch?v=PGUw2P55ZYM&list=PLc_vA1r0qoiQqCdWFDUDqI90oY5EjfGuO&index=5

Proton

- **对指定区域进行性能分析**

```
proton.start("matmul", hook="triton")
benchmark.run(show_plots=True, print_data=True)
proton.finalize()
```

- **proton profile数据可视化**

```
pip install llnl-hatchet
proton-viewer -m tflops/s,time/s <profile.hatchet>
```

Proton

- **用语义信息标注用户定义的区域**

```
with proton.scope(f"matmul_{M}_{N}_{K}"):
    if provider == "cublas":
        @proton.scope(
            "cublas",
            metrics={
                "flops": 2 * M * N * K,
                "bytes": (M * N + N * K + K * M) * a.element_size(),
            },
        )
        def cublas_matmul(a, b):
            torch.matmul(a, b)
        ......
    if provider == "triton":
        def enter_autotune(args, reset_only=False):
            if reset_only:
                return
            proton.enter_scope("<autotune>")

        def exit_autotune(args, exception):
            proton.exit_scope()

        matmul_kernel.pre_hook = enter_autotune
        matmul_kernel.post_hook = exit_autotune
        with proton.scope("triton"):
            ......
```

Proton

- **自定义元数据**

```
def metadata_fn(
    grid: tuple,
    metadata: NamedTuple,
    args: dict,
):
    grid_x, grid_y, grid_z = unpack_grid(grid)
    num_warps = metadata.num_warps
    num_stages = metadata.num_stages
    cluster_x, cluster_y, cluster_z = metadata.cluster_dims
    shared_memory = metadata.shared
    M, K = args["a_ptr"].shape
    K, N = args["b_ptr"].shape
    return {
        "name":
        f"matmul_<grid:{grid_x}x{grid_y}x{grid_z}>_<cluster:
{cluster_x}x{cluster_y}x{cluster_z}>_<warps:{num_warps}>_<shared:
{shared_memory}>_<stages:{num_stages}>",
        "flops": 2 * M * N * K,
        "bytes": (M * N + N * K + K * M) * args["a_ptr"].element_size(),
    }
@triton.jit(launch_metadata=metadata_fn)
def matmul_kernel(...):
    ......
```

课程总结

1、掌握TorchBench模型基准测试框架的使用

使用Torchbench运行模型生成Triton Kernel和profiler追踪文件

2、掌握编译模块性能的研究分析方法

使用-p选项单独研究每个编译模块的性能，观察终端输出信息的含义

3、掌握profiler追踪文件可视化查看分析方法

使用谷歌（或EDGE）浏览器查看整网和-p生成的.json profiler文件，观察各个可视化条目的含义并进行对比

4、掌握torch.profiler接口的使用方法

使用torch.profiler接口单独分析Kernel，尝试不同的参数选项功能

5、(可选) 了解其他分析工具HTA、Nsight Systems、Nsight Compute、Triton Proton等的安装和使用

先进编译实验室 Advanced Compilation Laboratory